

Procesamiento del Lenguaje Natural

Word Vectors

Jorge Ortiz Fuentes

29 de abril de 2025

Contenido

Introducción a Word Vectors

Vectores distribucionales

Vectores distribuidos (word embeddings)

Word2Vec

GloVe

FastText

Analogías y evaluación

Relación entre enfoques basados en conteo y predictivos

Herramientas

Introducción a Word Vectors

Recordatorio: vectores basados en conteo (BoW/TF-IDF)

Limitaciones conocidas

- Representan palabras/documentos basados en conteos
- Generan vectores **muy largos** (dimensión = |Vocabulario|) y **dispersos** (muchos ceros)
- **Problema fundamental:** no capturan el *significado* o la *similitud semántica* entre palabras individuales

¿Qué pasa con la similitud coseno?

- La similitud coseno entre vectores BoW/TF-IDF mide el solapamiento de términos, no necesariamente la similitud semántica intrínseca.

Word Vectors: una alternativa a la dispersión

¿Qué son los word vectors?

Son una forma de representar palabras como vectores

- El objetivo principal es capturar el **significado** de las palabras y las **relaciones semánticas** entre ellas.
- Pueden ser **densos** (pocos o ningún cero) o **dispersos**, dependiendo del método de creación. Los vectores **densos (embeddings)** son el enfoque predominante.
- Se aprenden automáticamente a partir de grandes cantidades de texto (corpus) mediante técnicas que modelan el contexto de las palabras.

Hacia un espacio vectorial semántico

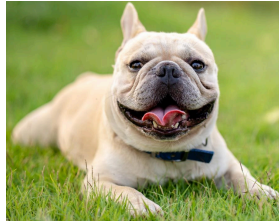
- **Idea clave:** La posición de una palabra en este nuevo espacio vectorial *codifica* sus propiedades semánticas y sintácticas.
- Palabras con significados similares tendrán vectores cercanos en este espacio.
- Por ejemplo, la **similitud coseno** entre los vectores de *rey* y *reina* será alta, reflejando su relación semántica.
- Estos vectores son fundamentales en las arquitecturas modernas de NLP basadas en redes neuronales.

Vectores distribucionales

¿Cómo definimos 'perro'?



El desafío de definir 'por lo que es'



¿Cómo asignamos significado?

La idea Saussureana: significado por oposición

- Ferdinand de Saussure, lingüista suizo, propuso que el significado no es inherente a los signos, sino que surge de su relación con otros signos dentro de un sistema
- **Valor negativo:** el significado se define por lo que *no* es
- Ejemplo: ¿Qué es un 'perro'? Solo podemos definirlo en contraste con otros conceptos (no es un gato, no es un lobo, no es un mueble, etc.)
- El significado es relacional y diferencial, no una propiedad positiva intrínseca

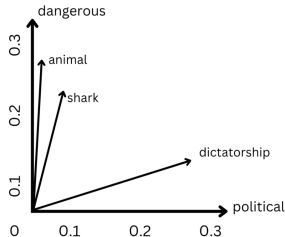
Principio fundamental

- **Hipótesis Distribucional** [[Harris, 1954](#)]: palabras que aparecen en los mismos **contextos** tienden a tener significados similares.
- “Una palabra se caracteriza por la **compañía** que mantiene” [[Firth, 1957](#)].
- “El significado de una palabra es su **uso** en el lenguaje” [[Wittgenstein, 1953](#)].

Representaciones distribucionales

Idea central

- Basado en la hipótesis distribucional, podemos representar palabras mediante vectores que capturen sus contextos típicos
- Estos vectores se denominan **representaciones distribucionales**
- La idea es que la similitud entre estos vectores refleje la similitud semántica entre las palabras



Construcción

Una forma posible es usar **vectores de alta dimensionalidad** donde cada dimensión corresponde a alguna medida de asociación con un contexto específico (ej. otra palabra, un documento).

- Una forma de construir vectores distribucionales es mediante las matrices palabra-contexto M
- Cada celda (i, j) representa la asociación entre una **palabra objetivo** w_i y un **contexto** c_j obtenido de un corpus
- Los contextos se definen comúnmente como ventanas de palabras que rodean a w_i

Matrices palabra-contexto: parámetros y observaciones

Parámetros importantes

- La longitud de la ventana k (entre 1 y 8 palabras a ambos lados de w_i)
- Si el vocabulario de palabras objetivo y de contexto es el mismo, M tiene dimensionalidad $|\mathcal{V}| \times |\mathcal{V}|$

Observación clave

Ventanas más cortas capturan información **sintáctica** (por ejemplo, POS), mientras que ventanas más largas capturan similitud **semántica**.

Ejemplo de vectores distribucionales con ventanas de contexto de tamaño 1

Ejemplo ilustrativo

A continuación, presentamos un ejemplo con las siguientes oraciones:

- I like deep learning
- I like NLP
- I enjoy flying

Este ejemplo ilustra cómo se construyen vectores distribucionales usando ventanas de contexto de tamaño 1, donde cada dimensión representa la frecuencia de co-ocurrencia con otra palabra del vocabulario

Ejemplo de vectores distribucionales con ventanas de contexto de tamaño 1

- I like deep learning.
- I like NLP.
- I enjoy flying.

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Ejemplo tomado de:

<http://cs224d.stanford.edu/lectures/CS224d-Lecture2.pdf>

Vectores distribuidos (word embeddings)

Limitaciones de los métodos basados en conteo

- Los vectores distribucionales basados en conteo aumentan en tamaño con el vocabulario
- Almacenar explícitamente la matriz de co-ocurrencia puede consumir mucha memoria
- Algunos modelos de clasificación no escalan bien a datos de alta dimensionalidad

Solución: word embeddings

- La comunidad de redes neuronales prefiere usar **representaciones distribuidas** o **word embeddings**
- Son vectores densos de palabras de baja dimensionalidad entrenados con redes neuronales

Características de los word embeddings

Propiedades clave

- Las dimensiones no son directamente interpretables, representan características latentes de la palabra
- Capturan propiedades sintácticas y semánticas útiles
- Se han convertido en un componente crucial de las arquitecturas de redes neuronales para NLP

Nota importante

La idea fundamental: El significado de la palabra está distribuido sobre una combinación de dimensiones.

Enfoques para obtener word embeddings

Dos enfoques principales

1. **Capas de embedding:** usar una capa de embedding en una arquitectura de red neuronal específica para una tarea, entrenada con ejemplos etiquetados (ej. análisis de sentimiento).
2. **Word embeddings pre-entrenados:** crear una tarea predictiva auxiliar a partir de corpus no etiquetados (ej. predecir la siguiente palabra) en la que los word embeddings surgirán naturalmente de la arquitectura de la red neuronal.

Estrategia combinada

Inicializa la capa de embedding con embeddings pre-entrenados para aprovechar lo mejor del aprendizaje supervisado y no supervisado.

Modelos Populares de Word Embeddings

Modelos más utilizados

- **Word2Vec:** Utiliza las arquitecturas CBOW y Skip-Gram para aprender embeddings a partir de grandes corpus, capturando relaciones semánticas y sintácticas [[Mikolov et al., 2013](#)].
- **GloVe:** (Global Vectors) Se basa en estadísticas globales de co-ocurrencia para generar representaciones vectoriales que reflejan similitudes semánticas [[Pennington et al., 2014](#)].
- **FastText:** Extiende Word2Vec incorporando información de subpalabras, lo que mejora la representación de palabras poco frecuentes y morfológicamente complejas [[Bojanowski et al., 2016](#)].

Word2Vec

¿Qué es Word2Vec?

- Word2Vec es un paquete de software que implementa dos arquitecturas de redes neuronales para entrenar word embeddings.
- Fue introducido por Tomas Mikolov y colaboradores en Google (2013).
- Permite aprender representaciones vectoriales densas de palabras a partir de grandes corpus de texto.

Principales enfoques

- **Modelo Skip-gram:** Dada una palabra central, se predicen las palabras de su contexto.
- **Modelo Continuous Bag-of-Words (CBOW):** A partir de las palabras del contexto, se predice la palabra central.

Modelo Skip-gram: Conceptos básicos

Funcionamiento general

- Para cada posición t , toma la palabra central w_t .
- Para cada palabra de contexto c en la ventana $c_{1:k}$, entrena un clasificador independiente que comparte pesos.
- La capa de entrada es un vector one-hot de dimensión $|V|$; la capa oculta (proyección) es de dimensión d .
- No se concatenan los k vectores antes de la salida; cada par (w_t, c) produce su propia loss.

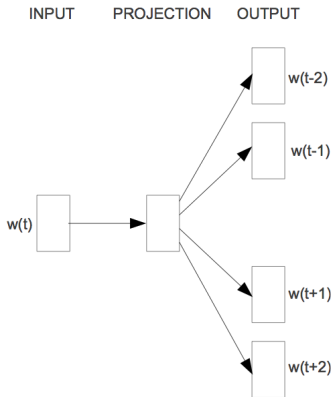
Representación inicial

Vectores one-hot en la entrada y en la salida antes de proyectar a d dimensiones.

Arquitectura del Modelo Skip-gram

Estructura de la red

- Compartición de pesos entre las k predicciones de contexto.
- Capa oculta de tamaño d (hiperparámetro, normalmente $d \ll |V|$).



Meta principal

- Aprender vectores $\vec{w}$ y $\vec{c}$ tales que cada palabra central prediga bien su contexto real.
- Maximizar la probabilidad conjunta de observar todos los pares (w_t, c) en el corpus D :

$$\sum_{(w,c) \in D} \log P(c \mid w)$$

Cálculo de $P(c \mid w)$: la función softmax

Definición

La probabilidad de observar una palabra de contexto c dada una palabra central w se calcula usando la función softmax:

$$P(c \mid w) = \frac{e^{\vec{c} \cdot \vec{w}}}{\sum_{c' \in V} e^{\vec{c}' \cdot \vec{w}}}$$

- $\sum_{c' \in V}$: normalización sobre todo el vocabulario V para obtener una distribución de probabilidad

Desafío computacional

Calcular el denominador requiere considerar *todas* las palabras del vocabulario V , lo cual es computacionalmente muy costoso si $|V|$ es grande (cientos de miles o millones).

Alternativas eficientes al softmax completo

El cálculo del denominador en la función softmax es computacionalmente prohibitivo para vocabularios grandes. Se pueden ocupar técnicas de optimización para hacerlo más eficiente:

- Negative sampling
- Hierarchical softmax

Estas técnicas aproximan la normalización completa o reducen la carga computacional de otras maneras.

Idea principal

Transforma el problema multiclase (predecir la palabra de contexto correcta entre $|V|$ opciones) en un conjunto de problemas de clasificación binaria.

- Para cada par real (w, c) , se considera una muestra positiva
- Se generan n muestras "negativas" (w, c')
- Se entrena un clasificador logístico (usando la función sigmoide) para distinguir entre la muestra positiva y las negativas
- El número de muestras negativas n es un hiperparámetro (usualmente 5-20)

Evita la costosa suma sobre todo el vocabulario V .

Estructura y funcionamiento

Utiliza un árbol binario para representar el vocabulario.

- Cada hoja del árbol corresponde a una palabra del vocabulario V
- Para calcular $P(c|w)$, se navega desde la raíz hasta la hoja correspondiente a c
- En cada nodo interno del camino, se toma una decisión binaria (izquierda o derecha) usando una función sigmoide y un vector asociado al nodo
- La probabilidad final $P(c|w)$ es el producto de las probabilidades sigmoides a lo largo del camino

Reduce la complejidad computacional de $O(|V|)$ a $O(\log |V|)$.

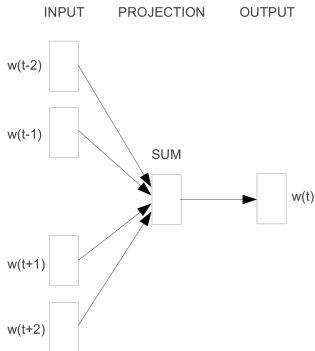
Skip-gram: Resumen y buenas prácticas

- Cada contexto se predice con un clasificador separado que comparte parámetros.
- Negative Sampling y Hierarchical Softmax hacen viable el entrenamiento en vocabularios grandes.
- Ajustar hiperparámetros clave:
 - Dimensión d de embeddings
 - Tamaño de ventana k
 - Número de negativos n
 - Tasa de subsampling para palabras frecuentes

Continuous Bag of Words (CBOW)

Enfoque alternativo

- Similar al modelo skip-gram pero ahora la palabra central se predice a partir del contexto circundante
- Invierte la dirección de predicción respecto al Skip-gram



CBOW

CBOW vs Skip-gram: ¿Cuándo usar cuál?

CBOW

- Predice la palabra central desde el contexto
- Más rápido de entrenar
- Mejor para palabras frecuentes

Skip-gram

- Predice el contexto desde la palabra central
- Más lento pero mejor con palabras raras
- Funciona bien con corpus pequeños

Recomendación general

Skip-gram suele funcionar mejor para la mayoría de las tareas, especialmente si el corpus no es masivo. CBOW es una alternativa más rápida si la velocidad es crítica.

GloVe

GloVe: vectores globales para palabras

Idea central

- GloVe crea una matriz de co-ocurrencias global para todo el corpus.
- Captura relaciones semánticas aprovechando estadísticas globales.

Enfoque híbrido

- Combina conteos globales (count-based) y aprendizaje predictivo.
- Usa una función de ponderación para suavizar el efecto de conteos extremos.
- A diferencia de Word2Vec, mira todo el corpus en lugar de sólo ventanas locales.

FastText

FastText: vectores enriquecidos con subpalabras

Idea central

- Extensión de Word2Vec que representa cada palabra como la suma de vectores de sus n-gramas.
- Entrena de forma muy eficiente usando skip-gram o CBOW con negative sampling.

Enfoque

- Descompone cada palabra en sub-componentes.
- Aprende vectores para cada n-grama y luego los agrupa para formar el embedding de la palabra.
- Captura información morfológica y mejora la calidad de embeddings para palabras raras o desconocidas.

FastText vs Word2Vec y GloVe

Fortalezas de FastText

- Capacidad para generar vectores para palabras desconocidas
- Buen rendimiento en lenguas morfológicamente ricas

Consideraciones

- Modelos más grandes (almacena vectores para n-gramas)
- Entrenamiento potencialmente más lento que Word2Vec
- La calidad puede depender de la elección de n-gramas

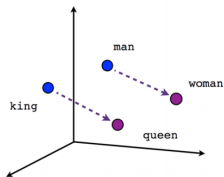
Analogías y evaluación

Relaciones semánticas

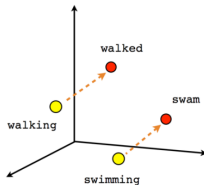
- Los word embeddings pueden capturar ciertas relaciones semánticas, por ejemplo, relaciones masculino-femenino, tiempo verbal y país-capital entre palabras
- Por ejemplo, se encuentra la siguiente relación para word embeddings entrenados con Word2Vec:

$$\vec{w}_{king} - \vec{w}_{man} + \vec{w}_{woman} \approx \vec{w}_{queen}$$

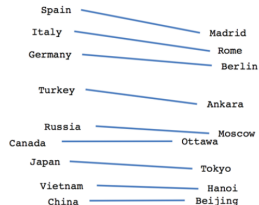
Analogías de palabras: ejemplo visual



Male-Female



Verb tense



Country-Capital

Fuente: <https://www.tensorflow.org/tutorials/word2vec>

Evaluación de word embeddings

Métodos de evaluación

- Existen muchos conjuntos de datos con asociaciones de pares de palabras anotadas por humanos o analogías de referencia que pueden usarse para evaluar algoritmos de word embeddings
- Estos enfoques se denominan *Enfoques de Evaluación Intrínseca*
- La mayoría están implementados en: <https://github.com/kudkudak/word-embeddings-benchmarks>

Evaluación extrínseca

Los word embeddings también pueden evaluarse extrínsecamente usándolos en una tarea externa de NLP (por ejemplo, etiquetado POS, análisis de sentimiento).

Relación entre enfoques basados en conteo y predictivos

Hipótesis distribucional compartida

- Tanto los **métodos basados en conteo** (p. ej. matrices palabra-contexto) como los **métodos predictivos** (p. ej. Word2Vec SGNS) parten de la misma idea: palabras que aparecen en contextos similares tienen significados similares.
- En ambos casos, la similitud entre vectores refleja la similitud semántica de las palabras.

Factorización implícita de PMI

- Levy y Goldberg demostraron que Skip-gram con Negative Sampling (SGNS) equivale a factorizar una matriz de *PMI* (Información Mutua Puntual) desplazada:

$$\text{SGNS} \approx \text{factorización de } (\text{PMI}_{w,c} - \log k)$$

- Con ello se unifica formalmente:
 - **Modelos basados en conteo:** construyen y factorizan PMI explícitamente
 - **Modelos predictivos (neuronales):** factorizan implícitamente esta matriz al optimizar la predicción de contexto
- En esencia, ambas familias extraen relaciones semánticas similares usando enfoques distintos

Herramientas

Gensim: Biblioteca para Word Embeddings

Descripción

Gensim es una biblioteca de código abierto en Python para procesamiento de lenguaje natural que implementa muchos algoritmos para entrenar word embeddings.

Enlaces útiles

- <https://radimrehurek.com/gensim/>
- <https://machinelearningmastery.com/develop-word-embeddings-python-gensim/>





Bojanowski, P., Grave, E., Joulin, A., and Mikolov, T. (2016).

Enriching word vectors with subword information.

arXiv preprint arXiv:1607.04606.



Firth, J. R. (1957).

A synopsis of linguistic theory, 1930-1955.

In *Studies in linguistic analysis*, pages 1–32. Blackwell.



Harris, Z. (1954).

Distributional structure.

Word, 10(23):146–162.



Levy, O. and Goldberg, Y. (2014).

Neural word embedding as implicit matrix factorization.

In *Advances in neural information processing systems*, pages 2177–2185.



Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013).

Distributed representations of words and phrases and their compositionality.

In Burges, C., Bottou, L., Welling, M., Ghahramani, Z., and Weinberger, K., editors, *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.



Pennington, J., Socher, R., and Manning, C. D. (2014).

Glove: Global vectors for word representation.

In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL, pages 1532–1543.



Wittgenstein, L. (1953).

Philosophical investigations.

Blackwell.